

EE5332 Mini Project Report

Matrix-Vector Multiplication

Ishan Aji Hassan (EE23B097)
Arjun Krishnaswamy (EE23B009)
Sai Harshith Gajendra (EE23B069)

1 Problem Description

Compute:

$$y = A \cdot x$$

where A is a matrix (up to 64×64) loaded via AXI-Lite, and x is streamed using AXI-Stream. The output vector y is streamed out.

2 Baseline Implementation

The baseline implementation consists of a straightforward double nested loop. Fixed-point arithmetic (`ap_fixed<24,14>`) is used, and the results were verified against the floating point reference.

3 HLS Optimizations

Three design variants were explored:

- Baseline implementation without pragmas

Baseline Synthesis Report

Timing Estimate

Target	Estimated	Uncertainty
10.00 ns	5.727 ns	2.70 ns

Performance & Resource Estimates

Modules

Loops

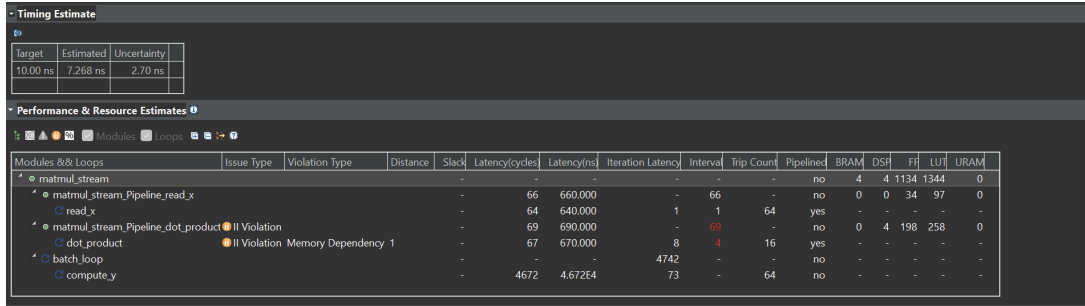
Modules && Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
matmul_stream				-	-	-	-	-	-	no	5	1	576	820	0
matmul_stream_Pipeline_read_x				-	66	660.000	-	66	-	no	0	0	9	72	0
read_x				-	64	640.000	1	1	64	yes	-	-	-	-	-
batch_loop				-	-	-	20743	-	-	no	-	-	-	-	-
compute_y				-	20672	2.070E5	323	-	64	no	-	-	-	-	-

Figure 1: Synthesis report for baseline implementation with `ap_fixed<16,2>`

Resource Analysis:

- **DSP Usage:** Only 1 DSP is used here, as expected with pipelining turned off. This DSP is re-used in a folded type architecture for computing each element of output vector, causing each loop of compute_y to have 323 cycle latency(meaning each inner loop takes around $323/(64 \text{ multiplications}) = 5$ cycles which is reasonable when accounting for memory access too.
- **BRAM Utilization:** 5 BRAMs are used. This is for storing the unpartitioned A matrix and x.local(input) vector. I used `ap_fixed<16,2>` for this synthesis, so there are $16*64*64 = 65,536$ bits to be stored for A. So it takes 65,536 bits/18 kilobits = around 4 BRAMs. x.local also uses 1 BRAM(because no pragmas are assigned to it, it defaults to using a BRAM).
- **FF Usage:** Very few FFs are used here. They are used for AXI logic and also for FSM for folded multiplier architecture.
- Loop unrolling with factor 4, and with ii=4 for inner product loop.

Loop Unrolling Synthesis Report



The screenshot shows the 'Timing Estimate' and 'Performance & Resource Estimates' sections of a synthesis report. The 'Timing Estimate' table shows a target of 10.00 ns, an estimated time of 7.268 ns, and an uncertainty of 2.70 ns. The 'Performance & Resource Estimates' table provides a detailed breakdown of resource usage across various modules and loops.

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
matmul_stream															
matmul_stream_Pipeline_read_x					66	660.000		66		no	4	4	1134	1344	0
read_x					64	640.000	1	1	64	yes	0	0	34	97	0
matmul_stream_Pipeline_dot_product	II Violation	Memory Dependency 1			69	690.000		69		no	0	4	198	258	0
dot_product					67	670.000	8	4	16	yes	-	-	-	-	-
batch_loop					-	-	4742	-	-	no	-	-	-	-	-
compute_y					4672	4.672E4	73	-	64	no	-	-	-	-	-

Figure 2: Synthesis report for loop unrolling with factor 4 and `ap_fixed<16,2>`

Resource Analysis:

- **DSP Usage:** I gave UNROLL directive inside inner dot product loop with a factor 4, resulting in 4 DSPs used instead of the 1 DSP used in baseline case. This also speeds up latency of compute_loop by a factor of approximately 5 (because 4 multiplications/4 ii cycles = 1 multiplication/cycle here as compared to 5 cycles/-multiplication for baseline).
- **BRAM Utilization:** Matrix A is cyclically partitioned with factor 4 in this implementation. Thus taking up same 4 BRAMs as in baseline case, except that the ordering of elements will be cyclical. x.local uses cyclical partitioning with factor 4 also, but uses FFs and LUTs instead of BRAM due to the Pragma.

I could not achieve ii=1 here despite partitioning both these arrays. Couldn't exactly figure out why this was happening. But it achieved ii=4 and cut down latency to 1/4 of baseline still.

- **Logic (LUT and FF):** Slightly Higher number of FFs and LUTs are used here compared to baseline because of partitioning of x_local, which uses FFs and LUTs to build a RAM to store it.
- Fully unrolled implementation with loop pipelining ($II = 1$)

Pipelined Design Synthesis Report

Target	Estimated	Uncertainty
10.00 ns	7.234 ns	1.25 ns

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
matmul_stream				-	-	-	-	-	-	no	128	64	15525	13483	0
matmul_stream_Pipeline_read_x				-	66	660.000	-	66	-	no	0	0	1570	673	0
read_x				-	64	640.000	1	1	64	yes	-	-	-	-	-
matmul_stream_Pipeline_compute_y				-	99	990.000	-	99	-	no	0	64	6884	6741	0
compute_y				-	97	970.000	35	1	64	yes	-	-	-	-	-
batch_loop				-	-	-	170	-	-	no	-	-	-	-	-

Figure 3: Synthesis report for pipelined design with `ap_fixed<24,14>`

Target	Estimated	Uncertainty
10.00 ns	6.534 ns	2.70 ns

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
matmul_stream				-	-	-	-	-	-	no	64	64	10873	7956	0
matmul_stream_Pipeline_read_x				-	66	660.000	-	66	-	no	0	0	1058	673	0
matmul_stream_Pipeline_compute_y				-	132	1.320E3	-	132	-	no	0	64	3256	1214	0
batch_loop				-	-	-	203	-	-	no	-	-	-	-	-

Figure 4: Synthesis report for pipelined design with `ap_fixed<16,2>`

Resource Analysis:

- **DSP Usage:** 64 DSPs are used for doing 64 parallel multiplications when calculating a single element of the output vector. This is because $II=1$ has been targeted for `compute_y` loop.
- **BRAM Utilization:** The BRAM usage is 128 because each element in A vector is fixed as `ap\_fixed<24,14>` and storing $64(\text{elements/column}) \times 24$ bits had made it so that a single column of A required 2 BRAMs to store, taking total BRAM usage to $2 \times 64(\text{columns of A}) = 128$. This has been reduced to 64 BRAMs total when switching A to use `ap_fixed<16,2>`.
- **FF Usage:** FF usage is within expected range. Flip-flops are needed to pipeline the adders for each iteration of `compute_y` loop to achieve $II=1$. They are also needed for storing partitioned input vector (around $64 \times 16 = 1024$ FFs), and also for AXI-Lite and for the Control FSM.

4 FPGA Implementation

The hardware implementation involved:

- Synthesizing the design using Vitis HLS
- Exporting the generated IP to Vivado
- Creating the block design using a TCL script (based on Lab 5 procedure)

The Vivado block design TCL script is shown below:

```
1 # ----- Configuration -----
2 set PROJ_NAME      "matmul"
3 set PROJ_DIR       "./build"
4 set PART           "xc7z020sclg400-1"
5 set MATMUL_IP      "./ip/matmul_stream"
6
7 # ----- Create Project -----
8 create_project ${PROJ_NAME} ${PROJ_DIR}/${PROJ_NAME} -part ${PART}
9   -force
10 set_property ip_repo_paths [list ${MATMUL_IP}] [current_project]
11 update_ip_catalog
12
13 # ----- Create Block Design -----
14 create_bd_design "matmul_design"
15
16 # -- Zynq PS --
17 create_bd_cell -type ip -vlnv xilinx.com:ip:processing_system7
18   :5.5 ps7
19 apply_bd_automation \
20   -rule xilinx.com:bd_rule:processing_system7 \
21   -config {make_external "FIXED_IO, DDR" Master "Disable" Slave
22     "Disable"} \
23   [get_bd_cells ps7]
24
25 # Enable HPO for DMA memory access, 100MHz PL clock
26 set_property -dict [list \
27   CONFIG.PCW_USE_S_AXI_HPO {1} \
28   CONFIG.PCW_FPGA0_PERIPHERAL_FREQMHZ {100} \
29 ] [get_bd_cells ps7]
30
31 # -- AXI DMA --
32 create_bd_cell -type ip -vlnv xilinx.com:ip:axi_dma:7.1 axi_dma_0
33 set_property -dict [list \
34   CONFIG.c_include_sg {0} \
35   CONFIG.c_sg_include_stscntrl_strm {0} \
36   CONFIG.c_mm2s_burst_size {256} \
37   CONFIG.c_s2mm_burst_size {256} \
38   CONFIG.c_sg_length_width {26} \
39 ] [get_bd_cells axi_dma_0]
40
41 # -- matmul_stream HLS IP --
```

```

39 create_bd_cell -type ip -vlnv xilinx.com:hls:matmul_stream:1.0
    matmul_stream_0
40
41 # ----- Stream Connections -----
42 connect_bd_intf_net \
43     [get_bd_intf_pins axi_dma_0/M_AXIS_MM2S] \
44     [get_bd_intf_pins matmul_stream_0/in_stream]
45
46 connect_bd_intf_net \
47     [get_bd_intf_pins matmul_stream_0/out_stream] \
48     [get_bd_intf_pins axi_dma_0/S_AXIS_S2MM]
49
50 # ----- AXI-Lite Control -----
51 apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
52     -config {
53         Master "/ps7/M_AXI_GP0" Slave "/axi_dma_0/S_AXI_LITE"
54     } [get_bd_intf_pins axi_dma_0/S_AXI_LITE]
55
56 apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
57     -config {
58         Master "/ps7/M_AXI_GP0" Slave "/matmul_stream_0/
59             s_axi_control"
60     } [get_bd_intf_pins matmul_stream_0/s_axi_control]
61
62 # ----- DMA Memory Ports -----
63 apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
64     -config {
65         Master "/axi_dma_0/M_AXI_MM2S" Slave "/ps7/S_AXI_HP0"
66     } [get_bd_intf_pins ps7/S_AXI_HP0]
67
68 apply_bd_automation -rule xilinx.com:bd_rule:axi4 \
69     -config {
70         Master "/axi_dma_0/M_AXI_S2MM" Slave "/ps7/S_AXI_HP0"
71     } [get_bd_intf_pins axi_dma_0/M_AXI_S2MM]
72
73 # ----- Finalize -----
74 regenerate_bd_layout
75 validate_bd_design
76 save_bd_design

```

Listing 1: Vivado Block Design TCL Script

5 Pynq Evaluation

Setup and Data Flow

The test procedure is as follows:

1. A random 64×64 matrix A and a batch of 10 random input vectors were generated in Python using NumPy, with values drawn uniformly from $[1, 10]$.

2. Both A and the input vectors were converted to `ap_fixed<24,14>` fixed-point representation by scaling with 2^{10} and casting to `int32`.
3. The matrix A was loaded into the IP's internal BRAM via AXI-Lite writes (`offset 0x0100`), and the scalar parameters `rows`, `cols`, and `num_vectors` were written to their respective control registers.
4. The flattened input batch was transferred to the accelerator via AXI-DMA (MM2S channel), and the output was received back into a contiguous output buffer (S2MM channel).
5. The raw 32-bit integers returned by the hardware were sign-extended from 24 bits (using arithmetic left/right shift by 8) and then divided by 2^{10} to recover the floating-point result.

The Python code used on the PYNQ board is shown below:

```

1 MATRIX_SIZE = 64
2 NUM_VECTORS = 10
3 FRACTIONAL_BITS = 10
4
5 # Generate random test data (float32)
6 A_float = np.random.uniform(1, 10, size=(MATRIX_SIZE, MATRIX_SIZE
7     )).astype(np.float32)
8 X_float = np.random.uniform(1, 10, size=(NUM_VECTORS, MATRIX_SIZE
9     )).astype(np.float32)
10
11 # Convert to fixed-point (ap_fixed<24,14> -> int32)
12 A_fixed = (A_float * (2**FRACTIONAL_BITS)).astype(np.int32)
13 X_fixed = (X_float * (2**FRACTIONAL_BITS)).astype(np.int32)
14
15 # Configure AXI-Lite control registers
16 matmul_ip.write(0x10, MATRIX_SIZE) # rows
17 matmul_ip.write(0x18, MATRIX_SIZE) # cols
18 matmul_ip.write(0x20, NUM_VECTORS) # num_vectors
19 matmul_ip.write(0x0100, A_fixed.flatten('F').tobytes()) # load A
20     into BRAM
21
22 # Allocate DMA buffers and run
23 in_buf = allocate(shape=(NUM_VECTORS * MATRIX_SIZE,), dtype=np.
24     int32)
25 out_buf = allocate(shape=(NUM_VECTORS * MATRIX_SIZE,), dtype=np.
26     int32)
27 np.copyto(in_buf, X_fixed.flatten())
28
29 matmul_ip.write(0x00, 0x01) # ap_start
30 t0 = time.perf_counter()
31 dma.sendchannel.transfer(in_buf)
32 dma.recvchannel.transfer(out_buf)
33 dma.sendchannel.wait()
34 dma.recvchannel.wait()
35 hw_time = time.perf_counter() - t0

```

```

31
32 # Decode fixed-point output (sign-extend 24-bit -> 32-bit)
33 Y_raw = np.array(out_buf).reshape(NUM_VECTORS, MATRIX_SIZE)
34 Y_raw = np.right_shift(np.left_shift(Y_raw, 8), 8)
35 Y_hw = Y_raw.astype(np.float32) / (2**FRACTIONAL_BITS)
36
37 # Golden reference on ARM CPU
38 t_cpu0 = time.perf_counter()
39 Y_ref = X_float @ A_float.T
40 cpu_time = time.perf_counter() - t_cpu0

```

Listing 2: PYNQ Evaluation Notebook

Results

The results from the on-board run are summarised in Table 1.

Table 1: PYNQ Evaluation Results (64×64 matrix, 10 vectors)

Metric	Value
Verification status	PASS
Max absolute error	4.57×10^{-1}
ARM CPU time	1.39 ms
FPGA + DMA time	3.01 ms
Speedup (FPGA vs. CPU)	0.46×
Throughput	3,323 vectors/s

6 Contributions

1. Ishan - C code, 1 pareto point, basic python notebook
2. Arjun and Harshith - Vivado tcl script, 1 pareto point, deployment on fpga board, complete the python notebook

We made use of the code and the procedure followed during the fir filter lab experiments and also used the help of LLMs (used it to help us with the scripts and understand some of the issues we faced while creating the vivado project)